



PDF Download
3551349.3560438.pdf
29 December 2025
Total Citations: 38
Total Downloads: 1129

 Latest updates: <https://dl.acm.org/doi/10.1145/3551349.3560438>

RESEARCH-ARTICLE

How Readable is Model-generated Code? Examining Readability and Visual Inspection of GitHub Copilot

NASER AL MADI, Colby College, Waterville, ME, United States

Open Access Support provided by:
Colby College

Published: 10 October 2022

[Citation in BibTeX format](#)

ASE '22: 37th IEEE/ACM International
Conference on Automated Software
Engineering
October 10 - 14, 2022
MI, Rochester, USA

How Readable is Model-generated Code?

Examining Readability and Visual Inspection of GitHub Copilot

Naser Al Madi
nsalmadi@colby.edu
Colby College
Waterville, Maine, USA

ABSTRACT

Background: Recent advancements in large language models have motivated the practical use of such models in code generation and program synthesis. However, little is known about the effects of such tools on code readability and visual attention in practice.

Objective: In this paper, we focus on GitHub Copilot to address the issues of readability and visual inspection of model generated code. Readability and low complexity are vital aspects of good source code, and visual inspection of generated code is important in light of automation bias.

Method: Through a human experiment ($n=21$) we compare model generated code to code written completely by human programmers. We use a combination of static code analysis and human annotators to assess code readability, and we use eye tracking to assess the visual inspection of code.

Results: Our results suggest that model generated code is comparable in complexity and readability to code written by human pair programmers. At the same time, eye tracking data suggests, to a statistically significant level, that programmers direct less visual attention to model generated code.

Conclusion: Our findings highlight that reading code is more important than ever, and programmers should beware of complacency and automation bias with model generated code.

CCS CONCEPTS

• **Software and its engineering** → **Development frameworks and environments**; • **Human-centered computing** → *Collaborative and social computing systems and tools*.

KEYWORDS

GitHub, Copilot, Readability, Eye Tracking, Empirical Study

ACM Reference Format:

Naser Al Madi. 2022. How Readable is Model-generated Code? Examining Readability and Visual Inspection of GitHub Copilot. In *37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*, October 10–14, 2022, Rochester, MI, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3551349.3560438>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).
ASE '22, October 10–14, 2022, Rochester, MI, USA

© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9475-8/22/10...\$15.00
<https://doi.org/10.1145/3551349.3560438>

1 INTRODUCTION

Artificial Intelligence (AI) has been used in software development in general and in generating code in particular since the 1970's [4, 12]. Recent advances in Large Language Models, such as GPT-2 [16] and GPT-3 [6], have motivated the use of such models in code generation and program synthesis [7, 11]. GitHub Copilot¹ is a software development tool that offers code generation of lines, code chunks, or even entire programs based on existing code and comments [22]. Copilot is built with the GPT-3 language model [7] and trained with large sets of public code. Copilot is marketed as an AI powered pair-programmer, a software development practice where two programmers collaboratively write a single piece of code.

Pair programming matches two programmers with similar or different levels of experience to work together on writing code. Programmers periodically switch between two roles, driver and navigator. The driver controls the mouse and keyboard and writes code while the navigator observes the driver's work and critically thinks about defects, structural issues, and alternative solutions, while looking at the larger picture of the code [1, 26]. This practice has been shown to increase code quality and enjoyment during the problem solving process [26].

Several studies [14, 22, 24, 28] have examined aspects of productivity, usability, and security of Copilot generated code. However, little is known about the effects of such tools on code readability and visual attention in practice. In addition to the importance of generating readable code, automation bias could lead some programmers to accept suggested code without inspection, resulting in defects that are discovered at a latent stage of the software development process, where fixing defects is significantly more costly.

Therefore, in this paper we aim to study GitHub Copilot empirically with eye tracking in a natural software development environment (VS Code IDE). We focus on the issues of code readability and visual inspection of Copilot generated code. Through an experiment with 21 participants we compare code written with Copilot to code written by human pair-programmers (as driver and navigator). Using static code analysis, human readability annotators, and eye tracking we aim to answer the following research questions:

- *RQ1: How readable is Copilot generated code compared to code written completely by human programmers?*
- *RQ2: How well do programmers inspect Copilot generated code?*

Our results have implications to Copilot users and creators, highlighting the importance of reading and inspecting model generated code and being cautious of automation bias.

¹<https://copilot.github.com/>

2 EXPERIMENT

In this section, we present the details of our **IRB approved** experiment and the sources of data we used to answer our research questions. Our within-subject study consists of 21 participants in total, with data from 15 participants used in the eye tracking analysis. We make the our data and replication package publicly available through the following Open Science Framework [LINK].

2.1 Participants

The 21 participants in our study were Computer Science students with experience ranging from first year to fourth year. Some participants are in their first Computer Science course (CS1), while other students have completed several internships as professional programmers. The programming experience of participants ranged from 1 to 8 years, and 12 out of 21 participants were familiar with pair-programming. All participants were above the age of 18, and 8 were Female, 12 Male, and 1 Non-binary. Participation was voluntary, and participants were awarded a \$10 gift card after the experiment. Also, some eye tracking recordings for participants who wear corrective glasses were corrupt or had error beyond 1.5 degrees, therefore they were dropped from the eye tracking data analysis (their code is still used in the code analyses). Therefore, the eye tracking analysis is based on data from 15 participants.

2.2 Apparatus

The experiment lab consisted of a pair-programming station with two screens, two keyboards, and two mice connected to a single computer. The eye movement of the participant is tracked with an EyeLink 1000 Plus (SR Research) eye tracker, with a sampling frequency of 1000 samples per second. The screen resolution was set to 1920x1080. The development environment is Visual Studio Code (VS Code) with Copilot add-on that could be activated or deactivated between trials.

2.3 Procedure

First, the participant is entered into the experiment lab, and the general procedure of the experiment is explained along with an online consent form. If the participant agrees to proceed with the experiment, the online form collects the participant's programming experience in years, demographics, and whether they had prior experience with pair-programming or not. Regardless of the participant's experience with pair-programming, every participant watches a video explaining pair-programming and the roles of driver and navigator and how they typically alternate.

Participants worked on developing text-based Minesweeper game in Python. None of the participants had implemented this game before, and the participants were familiarized with the rules of the game by playing several rounds of the game prior to the recorded development task. After that, the participant's seat is adjusted for comfort along with a chin-rest that guarantees that the participant is captured by the eye tracker for the full duration of the experiment. An eye tracker calibration and validation steps are taken to guarantee that eye movement is being tracked and recorded accurately on the computer screen.

The development task was done under three conditions in randomized order. The conditions are pair programming with Copilot,

pair programming as a driver with another human volunteer, and pair programming as a navigator with another human volunteer. The time allocated is 20 minutes for Copilot, 10 minutes as a driver, and 10 minutes as a navigator (20 minutes with a human and 20 minutes with Copilot). The order of the three trial conditions was randomized to prevent the experience effect from influencing the results. The experience effect describes how participants perform better after each trial as they gain experience with the task and environment.

3 METHODS

3.1 Static Code Analyses and Readability:

Despite the widespread use of static code analyses metrics [13], depending completely on such metrics can be problematic [10]. Static code metrics are heuristics that often fail to measure what they aim to measure [19]. Nonetheless, a recent study by Peitek et al. found that some source code metrics correlate with programmer cognition as measured by fMRI [15].

Therefore, we cautiously combine static code metrics with readability assessment by human annotators to gain insights on the complexity and readability of source code. To answer our research questions, we collect the following metrics from each trial using Radon² and Pycodestyle³:

- Code Size: Logical Lines Of Code (LLOC).
- Complexity: Cyclomatic Complexity.
- Maintainability: Microsoft Visual Studio Maintainability.
- Vocabulary: Halstead Vocabulary.
- Volume: Halstead Volume.
- Difficulty: Halstead Difficulty.
- Coding style adherence: Number of coding style errors according to PEP8 coding style guide for Python [25].

We divide the number of coding style errors in each trial by the number of lines of code in the trial to calculate normalized coding style errors per line of code. This normalization accounts for the variations in the number of lines of code added in each trial, without influence from how much code was written in the trial. The idea here is that comparing the metrics of code written with Copilot to code written completely by human programmers, will allow us to see if Copilot code is more complex or substantially different from human written code.

In addition, we ask five programmers (two female, and three male) with two to five years of programming experience to annotate the readability of 20 code snippets from our experiment through an online survey. The code snippets are from Copilot, driver, and navigator trials, yet the origin of the code was hidden from the annotators. The annotators were given general written guidelines with examples of readable and unreadable code, and they were asked to assess the readability of each snippet on the scale: very unreadable, unreadable, neutral, readable, very readable. In addition, the annotators were asked to justify their readability assessment in writing.

²<https://pypi.org/project/radon/>

³<https://pypi.org/project/pycodestyle/>

3.2 Eye Tracking:

We developed our own tool for tracking eye movement in the VS Code IDE. Since existing tools for eye tracking in IDEs such as iTrace [21] do not yet support VS Code, and Copilot was only available on VS Code during the time of our experiment.

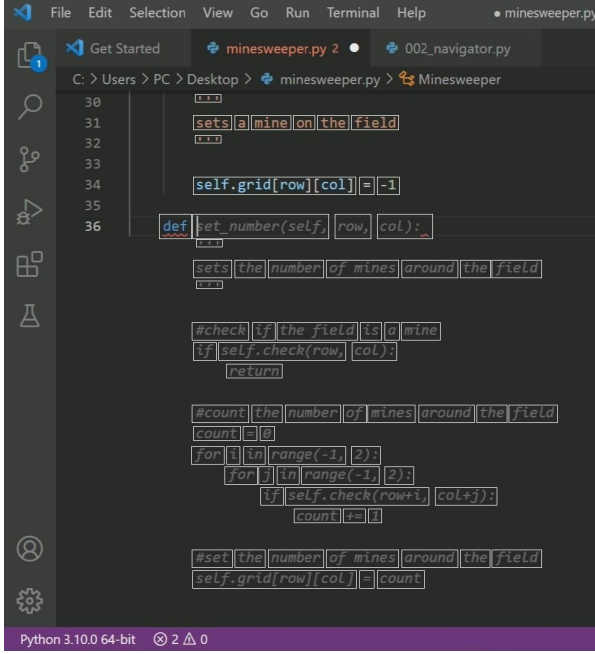


Figure 1: A sample program showing token level AOIs as generated by our tool for processing eye tracking data in VS Code. Code in grey is suggested by Copilot.

Our tool generates token-level Areas Of Interest (AOIs) as shown in Figure 1. With this, we can conduct a hit test to examine if a fixation was over a token or not using the Eye-Movement in Programming Toolkit (EMTK) [2]. This process is repeated until all the fixations in the experiment were analyzed. With this procedure, we were able to collect relevant eye movement data in a dynamic and natural coding environment (VS Code).

From fixation durations over AOIs we can calculate eye movement metrics that give us insights on the cognitive processes involved in reading code [3, 20]. We use the following metrics to compare eye movement during the Copilot trial to driver and navigator trials:

- Fixation Count: The total number of fixations on a token of code.
- Total Time: The sum of all fixation durations on a token of code (in millisecond).
- First-Fixation Duration: The duration of the first fixation on a token of code (in millisecond).
- Single-Fixation Duration: The duration of the fixation when only one fixation was made on the token (in millisecond).

The connection between eye movement and cognition has been well established in cognitive psychology [17], where First Fixation Duration has been connected to lexical access (word identification),

or the retrieval of word sound and meaning from memory [17, 18]. Total Time gives an indication of the time taken to complete lexical access and comprehension, where longer durations indicate more processing. These metrics have been used extensively in psychology and software engineering research [20], with some of the earliest studies using fixation count and fixation duration to find the AOIs that attract more attention [8, 9, 23]. In summary, we associate fixation count and longer fixations with more cognitive processing.

4 RESULTS

4.1 Readability Analyses

Our first research question (RQ1) focuses on the readability and complexity of GitHub Copilot generated code compared to code written by human programmers. To answer this question, we use static code analysis and a group of human annotators to assess the readability of code created in each condition (i.e., Copilot, Driver, Navigator).

Table 1: Comparing mean static code metrics of code written in each trial condition (i.e., Copilot, Driver, and Navigator). p shows the probability that Copilot, Driver, and Navigator data belong to the same population. Bold when $p < 0.01$.

	Copilot	Driver	Navigator	p
Code Size (LLOC)	19	11	15	.03
Cyclomatic complexity	3	2	2	.06
Maintainability	23	24	27	.96
Halstead vocabulary	13	9	12	.14
Halstead volume	222	149	142	.08
Halstead difficulty	3	1	2	.006
Style errors / LLOC	0.21	0.54	0.26	.08

We normalize the logical lines of code (LLOC) by the duration of the trials, therefore our numbers correspond to equal durations of each condition. Comparing mean code metrics in Table 1, we see that programmers on average wrote slightly more code during the Copilot trial, in addition the Halstead volume of Copilot code appears larger than code written during Driver and Navigator trials. Cyclomatic complexity, maintainability, and Halstead Vocabulary appears comparable between conditions.

On the other hand, Halstead difficulty is higher with Copilot and the number coding style errors per line of code is higher in the driver trial. Nonetheless, statistical tests show that only Halstead difficulty is statistically different between the three conditions. A Kruskal-Wallis test found statistically significant difference between the Halstead difficulty of Copilot, driver, and navigator codes ($H(2) = 10.13$, $p = .006$). Pairwise comparisons using Dunn's test indicated that Copilot Halstead difficulty scores were observed to be significantly different from those of the Driver code ($p = .004$). No other differences were statistically significant. The effect size

for this analysis ($d = .99$) was found to exceed Cohen's convention for a large effect ($d = .80$).

These results suggest that the code created during the Copilot trial is largely comparable to code written by human programmers during the driver and navigator trials. At the same time, the results suggest that code written during the Copilot trial appears to be more difficult to comprehend according to Halstead difficulty metric, on a statistically significant level.

Furthermore, we approach RQ1 and code readability from the angle of human annotators who assessed the readability of code snippet that were written during Copilot, Driver, or Navigator trials. Figure 2 shows a violin plot of readability assessment, where the white dot is the median, the upper and lower limits are the maximum and minimum values respectively, and the shape is a histogram of the data points. The figure shows comparable median values for readability, although some Copilot snippets received lower readability assessment by human annotators.

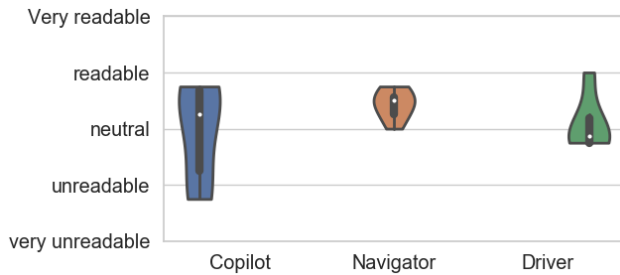


Figure 2: Code readability assessment by human annotators for code written during the Copilot, Driver, and Navigator trials.

Using a scale between 0 and 4 to represent readability scores between "very unreadable" to "very readable," we find that the mean readability score for Copilot is 2.1, for navigator is 2.1, and for driver 2.4. Nonetheless, A Kruskal-Wallis test found no statistically significant difference between the readability assessment for Copilot, driver, and navigator codes ($H(2) = 1.52, p = .46$).

Overall, The readability analyses results suggest that Copilot generated code is comparable in complexity and readability to code written entirely by human programmers. The only exception is Halstead difficulty metric, where copilot code appeared more difficult on a statistically significant level. We will reflect more on these results in the discussion section.

4.2 Eye Tracking Analyses

In this section, we focus on the eye movement of programmers while programming with Copilot compared with programming with a human programmer. Our focus in RQ2 is to examine how well do programmers inspect Copilot generated code? Our hypothesis is that code during a Copilot trial receives less scrutiny and inspection compared with code written with a human pair-programmer. Therefore, we examine Fixation Count, Total Time, First Fixation Duration, and Single Fixation Duration across trial conditions.

Analyzing the eye movement of programmers in each condition, we compare eye movement metrics during Copilot, driver, and

Table 2: Comparing eye movement metrics of programmers in each trial condition (duration in milliseconds). p shows the probability that Copilot, Driver, and Navigator data belong to the same population. Bold when $p < 0.01$.

Metric (Mean)	Copilot	Driver	Navigator	p
Fixation Count	2.10	2.38	2.33	<.001
Total Time	585	696	664	<.001
First Fixation	273	284	279	<.001
Single Fixation	282	292	281	.032

navigator trials in Table 2. The results show higher average fixation counts and higher fixation duration in the driver and navigator trials compared to the Copilot trial.

One-way ANOVA tests were performed to compare the effect of trial condition on eye movement metrics. The tests revealed that there was a statistically significant difference in Fixation Count, Total Time, and First Fixation duration between at least two groups ($p < .001$), as shown in Table 2. Tukey's HSD Test for multiple comparisons found that the mean values of Fixation Count, Total Time, and First Fixation Duration were significantly different between Copilot and Driver ($p < .001$), and Copilot and Navigator ($p < .001$). There was no statistically significant difference in Single Fixation Duration between Copilot, Driver, and Navigator ($p = .032$).

These results suggest that programmer's make fewer and shorter fixations while using Copilot, in comparison to pair-programming with a human. These results suggest that programmers direct less visual attention to Copilot generated code.

5 DISCUSSION

Our code analyses and readability annotation results in RQ1 suggest that code written with Copilot is comparable in complexity and readability to code written fully by human programmers. The only outlier to this is Halstead Difficulty, which was higher with Copilot, to a statistically significant level. These results match the results from Xu et al. which found no statistical difference in complexity between model generated code and code written by humans [27].

On the other hand, our eye tracking results of RQ2 suggest that programmers make fewer fixations and spend less time reading code during the Copilot trial. This might be an indicator of less inspection or over-reliance on AI (automation bias), as we have observed some participants accept Copilot suggestions with little to no inspection. This has been reported by another paper that studied Copilot [24]. In our experiment, we also observed that participants sometimes interact with large chunks of suggested code during a Copilot trial, and sometimes the suggested code requires scrolling up and down to view fully.

Although the productivity benefits of Copilot remain questionable [28], our results suggest that reading and inspecting model-generated code is important. Over-confidence in model generated code can lead to costly maintainability problems and issues in code quality. For model creators, there are a number of ideas to aid code readability when generating code. First, incorporating readability in the model itself to prioritize the most readable suggestion, if multiple suggestions are available. This requires new code readability metrics that match the readability assessment of human

programmers. Second, generating large chunks of code all at once possibly hinders code inspection, perhaps presenting shorter code suggestions more gradually would aid code inspection. Third, programmers spend more time reading code than writing code, in fact from an economic perspective code comprehension takes the largest proportion of software development time and cost [5]. Therefore applying large code models, such as Copilot Codex, to aid code comprehension could be very beneficial to the software engineering community.

6 THREATS TO VALIDITY

Internal validity: In regards to internal validity, it is typical for eye tracking studies to use static stimuli, and that does not match the real-life interactive experience of programming. Therefore, we created a custom tool for eye tracking in the VS Code IDE, with an interactive experience that matches the activities of real-life software development. To mitigate threats of eye tracker accuracy, we used a high frequency eye tracker (1000 samples per second) with a high spatial resolution, and we corrected drift using a universal offset where all fixations are moved together. In addition, we dropped recordings that had higher error than 1.5 degrees, to make sure our eye tracking data is reliable.

External validity: A threat to the generalization of our results is that our participants were students. This was partially mitigated by the inclusion of students with widely varying degrees of expertise, ranging from 1 year to 8 years of programming experience, yet we plan to include industry professionals in a future study.

Construct validity: Our code metrics were direct objective counts. The relationship between code metrics and cognition is still not established, therefore we cautiously use code metrics purely to compare Copilot code to human written code. In addition, we utilize human annotation to get a "ground truth" of readability to cover the gap between static code metrics and readability.

Conclusion validity: Every statistical test has the threat of producing a false positive or negative, and we attempted to mitigate this threat by applying Bonferroni correction when we perform multiple-comparisons. Nonetheless, as with every empirical experiment, multiple studies are needed to verify results, which we encourage by making our data and code publicly available.

7 FUTURE WORK

We plan to recruit a larger sample with professional developers from the industry in our experiment. In addition, we plan to include other human factors in the assessment of model-generated code. Eye tracking can also be used to assess cognitive load and other aspects of interaction between programmers and AI models.

REFERENCES

- [1] Ritchie Schacher Adam Archer and Scott Will. [n.d.]. *Program in Pairs*. Retrieved December 31, 2021 from https://www.ibm.com/garage/method/practices/code/practice_pair_programming/
- [2] Naser Al Madi, Drew Guarnera, Bonita Sharif, and Jonathan Maletic. 2021. EMIP Toolkit: A Python Library for Customized Post-processing of the Eye Movements in Programming Dataset. In *ACM Symposium on Eye Tracking Research and Applications*. 1–6.
- [3] Naser Al Madi, Cole S Peterson, Bonita Sharif, and Jonathan I Maletic. 2021. From Novice to Expert: Analysis of Token Level Effects in a Longitudinal Eye Tracking Study. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 172–183.
- [4] Marco Barenkamp, Jonas Rebstadt, and Oliver Thomas. 2020. Applications of AI in classical software engineering. *AI Perspectives* 2, 1 (2020), 1–15.
- [5] Boehm Barry et al. 1981. Software engineering economics. *New York* 197 (1981).
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Martha E Crosby, Jean Scholtz, and Susan Wiedenbeck. 2002. The Roles Beacons Play in Comprehension for Novice and Expert Programmers.. In *PPIG*. 5.
- [9] Martha E Crosby and Jan Stelovsky. 1990. How do we read algorithms? A case study. *Computer* 23, 1 (1990), 25–35.
- [10] Cem Kaner et al. 2004. Software engineering metrics: What do they measure and how do we know?. In *In METRICS 2004*. IEEE CS. Citeseer.
- [11] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-Level Code Generation with AlphaCode. (2022).
- [12] Zohar Manna and Richard J Waldinger. 1971. Toward automatic program synthesis. *Commun. ACM* 14, 3 (1971), 151–165.
- [13] Alberto S Nuñez-Varela, Héctor G Pérez-Gonzalez, Francisco E Martínez-Perez, and Carlos Soubervielle-Montalvo. 2017. Source code metrics: A systematic mapping study. *Journal of Systems and Software* 128 (2017), 164–197.
- [14] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2021. Can OpenAI Codex and Other Large Language Models Help Us Fix Security Bugs? *arXiv preprint arXiv:2112.02125* (2021).
- [15] Norman Peitek, Sven Apel, Chris Parnin, André Brechmann, and Janet Siegmund. 2021. Program comprehension and code complexity metrics: An fmri study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 524–536.
- [16] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [17] Keith Rayner. 1998. Eye movements in reading and information processing: 20 years of research. *Psychological bulletin* 124, 3 (1998), 372.
- [18] Erik D Reichle, Keith Rayner, and Alexander Pollatsek. 2003. The EZ Reader model of eye-movement control in reading: Comparisons to other models. *Behavioral and brain sciences* 26, 4 (2003), 445–476.
- [19] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2017. Automatically assessing code understandability: How far are we?. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 417–427.
- [20] Zohreh Sharafi, Bonita Sharif, Yann-Gaël Guéhéneuc, Andrew Begel, Roman Bednarik, and Martha Crosby. 2020. A practical guide on conducting eye tracking studies in software engineering. *Empirical Software Engineering* 25, 5 (2020), 3128–3174.
- [21] Bonita Sharif and Jonathan I Maletic. 2016. iTrace: Overcoming the Limitations of Short Code Examples in Eye Tracking Experiments.. In *ICSME*. 647.
- [22] Dominik Sobania, Martin Briesch, and Franz Rothlauf. 2021. Choose Your Programming Copilot: A Comparison of the Program Synthesis Performance of GitHub Copilot and Genetic Programming. *arXiv preprint arXiv:2111.07875* (2021).
- [23] Hidetake Uwano, Masahide Nakamura, Akito Monden, and Ken-ichi Matsumoto. 2006. Analyzing individual performance of source code review using reviewers’ eye movement. In *Proceedings of the 2006 symposium on Eye tracking research & applications*. 133–140.
- [24] Priyan Vaithilingam, Tianyi Zhang, and Elena Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI ’22 Extended Abstracts)*. ACM.
- [25] Guido Van Rossum, Barry Warsaw, and Nick Coghlan. 2001. PEP 8: style guide for Python code. *Python.org* 1565 (2001).
- [26] Laurie Williams. 2001. Integrating pair programming into a software development process. In *Proceedings 14th Conference on Software Engineering Education and Training. In search of a software engineering profession (Cat. No. PR01059)*. IEEE, 27–36.
- [27] Frank F Xu, Bogdan Vasilescu, and Graham Neubig. 2022. In-IDE Code Generation from Natural Language: Promise and Challenges. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 2 (2022), 1–47.
- [28] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.